

APIO 自転車

APIO 自転車は、APIO 市で新たに始まった自転車シェアリングサービスである。市中にいくつかのポートが設置されており、ユーザーは任意のポートで自転車を借りたり返したりすることができる。その便利さと価格の安さから、APIO 自転車はすぐに APIO 市で最も重要な交通手段となった。しかし、全ての自転車シェアリングサービスが向き合う問題に、APIO 自転車もまた直面してしまった。各ポート間で貸出数と返却数のバランスが取れなくなってしまったのだ。その結果、あるポートでは自転車の数がとても少なくなって近くの住民が借りられなくなってしまい、あるポートでは地元の住民が必要ないほどの数の自転車がたまってしまった。

APIO 自転車は、この問題に対処するため、再調整用のトラックを毎日夜に派遣してポートに自転車を再分配し、それぞれのポートに適切な数の自転車がある状態を維持できるようにすることを計画している。APIO 市には N 個のポートがあり、 $0, 1, \dots, N - 1$ の番号が付けられている。調査の結果、毎日夕方にポート i にある自転車の数はいつも固定の値 $A[i]$ であり、夜に間に自転車を借りたり返したりする人はいないことが、APIO 自転車により発見された。彼らの目的は、毎朝、ポート i にちょうど $B[i]$ 台の自転車があるようにすることである。

これらの N ポートの間には、再調整用トラックのための $N - 1$ 本の道路がある。それぞれの道路は 2 つの異なるポートを結んでおり、トラックはこれらの道路しか通ることができない。それぞれの道路は長さ 1 である。この道路網は連結であり、トラックはどの 2 つのポートの間もいくつかの道路を通して移動することができることが保証されている。毎夜、APIO 自転車は再調整用のトラックを派遣し、各ポート i にちょうど $B[i]$ 台の自転車があるようにしなければならない。トラックはどのポートから出発しても良く、またどのポートで移動を終えても良い。

トラックは再調整を始めるとき空である。運転手は、トラックがポートにいるときはいつでも、自転車をポートからトラックに積んだり、自転車をトラックからポートに降ろしたりすることができる。トラックとポートには何台の自転車があっても良いが、いずれの場所においても自転車の数が負にならない。彼らは再調整を完了するために必要な最小の移動距離と、それに対応する戦略を求めたがっている。

実装の詳細

あなたは以下の関数を実装する必要がある。

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A ：長さ N の配列で、 $A[i]$ は毎日夕方にポート i にある自転車の数である。
- B ：長さ N の配列で、 $B[i]$ は毎日朝にポート i にある自転車の数の目標とする値である。
- U, V ：長さ $N - 1$ の配列で、道路網の情報を表す。各 $0 \leq i < N - 1$ に対し、 i 番目の道路はポート $U[i]$ とポート $V[i]$ を結んでいる。
- この関数は各テストケースで**高々 150 000 回**しか呼ばれない。

関数は、同じ長さ $k + 1$ を持つ配列の組 (X, Y) を返さなければならない。これらの配列は、再調整の戦略を表している：

- X ：訪れるポートの番号を順に持つ配列である。
- Y ：訪れたそれぞれのポートでの自転車の積み降ろしの数を表す。各 $0 \leq j \leq k$ に対し、
 - $Y[j] \geq 0$ であれば、運転手は $Y[j]$ 台の自転車をトラックからポート $X[j]$ に降ろし、ポートにある自転車の数は $Y[j]$ 増える。
 - $Y[j] < 0$ であれば、運転手は $-Y[j]$ 台の自転車をポート $X[j]$ からトラックに積み、ポートにある自転車の数は $-Y[j]$ 減る。

距離 k を移動する**正当な再調整戦略** (X, Y) は、以下の条件を満たす必要がある：

- 各 $0 \leq j \leq k$ に対し、 $0 \leq X[j] < N$ 。
- 各 $0 \leq j < k$ に対し、ポート $X[j]$ とポート $X[j + 1]$ は直接道路で結ばれている。
- 各 $0 \leq j \leq k$ に対し、 $\sum_{t=0}^j Y[t] \leq 0$ 。
- 各ポート $0 \leq i < N$ と各ステップ $0 \leq j \leq k$ に対し、 $S_{i,j}$ を、 $0 \leq t \leq j$ かつ $X[t] = i$ を満たす各ステップ t についての $Y[t]$ の合計とする。
 - ポート i にある自転車の数は負にはならない： $A[i] + S_{i,j} \geq 0$ 。
 - 戦略の終了後、各ポート i にはちょうど $B[i]$ 個の自転車がなくてはならない： $A[i] + S_{i,k} = B[i]$ 。

制約

- $2 \leq N \leq 300\,000$ 。
- 各テストケースで、全ての `find_rebalancing_strategy` の呼び出しにおける N の合計は 300 000 を超えない。
- 各 $0 \leq i < N - 1$ に対し、 $0 \leq U[i], V[i] < N$ かつ $U[i] \neq V[i]$ 。
- 各 $0 \leq i < N$ に対し、 $0 \leq A[i], B[i] \leq 10^9$ 。

- 任意の 2 ポートの間をいくつかの道路を通して移動することができる.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- $0 \leq i < N$ かつ $A[i] \neq B[i]$ なる i が少なくとも 1 つ存在する.

小課題と採点

T を `find_rebalancing_strategy` の呼び出しの回数とし, $\sum N$ を全ての `find_rebalancing_strategy` の呼び出しにおける N の合計とする.

小課題	得点	追加の制約
1	4	ポートと道路は性質 P を満たす (下部を確認せよ). $0 \leq i < N$ かつ $A[i] > 0$ を満たす i がただ 1 つ存在する.
2	11	$T \leq 10$. $N \leq 7$. ポートと道路は性質 P を満たす.
3	16	ポートと道路は性質 P を満たす.
4	9	$0 \leq i < N$ かつ $A[i] > 0$ を満たす i がただ 1 つ存在する.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	追加の制約はない.

性質 P: 各 $0 \leq i < N - 1$ に対し, $U[i] = i$ かつ $V[i] = i + 1$. 各 $0 \leq i < N$ に対し, $A[i] \neq B[i]$.

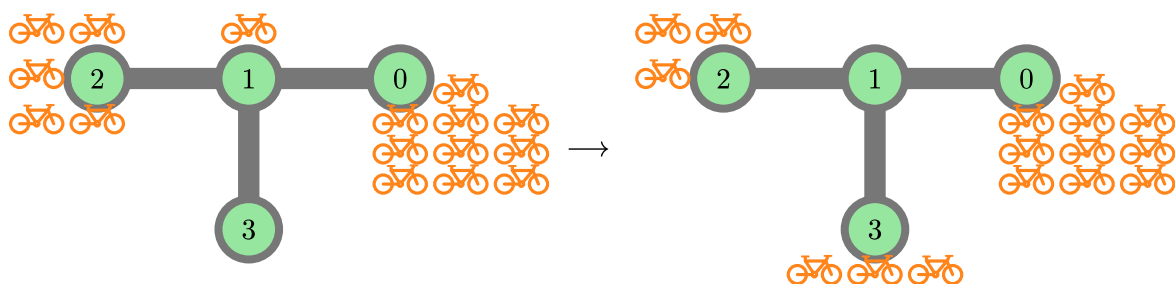
k^* を移動距離としてあり得る最小値として, 配列 X と Y がちょうど $k^* + 1$ の長さを持つが, (X, Y) が正当な戦略でない場合, あなたは得点の 50% を得る.

例

例 1

以下の呼び出しを考える:

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



APIO 市には $N = 4$ 個のポートがある．はじめ，ポートには $A = [10, 1, 5, 0]$ 台の自転車がおり，目標はそれぞれのポートに $B = [10, 0, 3, 3]$ 台の自転車があることである．

再調整用のトラックがポート 2 を出発し，以下のステップに従うことを考える：

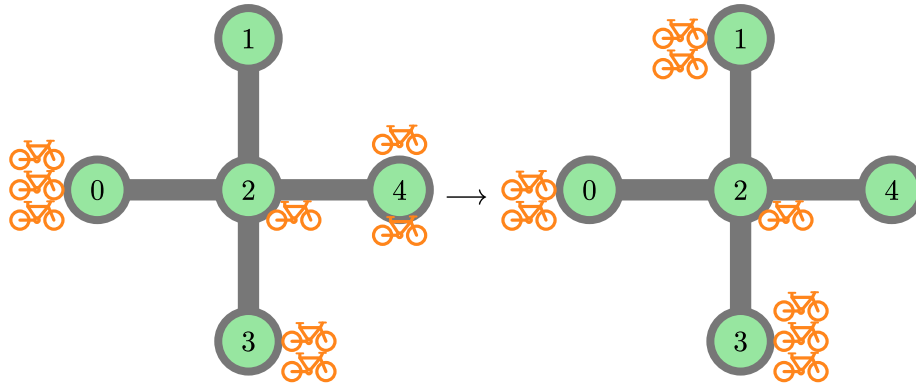
- ポート 2 で，2 台の自転車をトラックに積む．このときポート 2 には 3 台の自転車がおり，トラックには 2 台の自転車がある．
- ポート 1 に移動し，1 台の自転車をトラックに積む．このときポート 1 には 0 台の自転車がおり，トラックには 3 台の自転車がある．
- ポート 3 に移動し，3 台の自転車を降ろす．このときポート 3 には 3 台の自転車がおり，トラックには 0 台の自転車がある．

移動距離の合計は 2 である．この戦略に対応する返り値は $X = [2, 1, 3]$ と $Y = [-2, -1, 3]$ である．この戦略が最小の移動距離を達成することが証明できる．よって，関数は $([2, 1, 3], [-2, -1, 3])$ を返すべきである．

例 2

以下の呼び出しを考える：

```
find_rebalancing_strategy(5,
                          [3, 0, 1, 2, 2],
                          [2, 2, 1, 3, 0],
                          [2, 2, 2, 2],
                          [0, 4, 3, 1])
```



APIO 市には $N = 5$ 個のポートがある．はじめ，ポートには $A = [3, 0, 1, 2, 2]$ 台の自転車があり，目標はそれぞれのポートに $B = [2, 2, 1, 3, 0]$ 台の自転車があることである．

再調整用のトラックがポート 0 を出発し，以下のステップに従うことを考える：

- ポート 0 で，1 台の自転車を積む．
- ポート 2 に移動し，1 台の自転車を積む．
- ポート 1 に移動し，2 台の自転車を降ろす．
- ポート 2 に移動する．
- ポート 4 に移動し，2 台の自転車を積む．
- ポート 2 に移動し，1 台の自転車を降ろす．
- ポート 3 に移動し，1 台の自転車を降ろす．

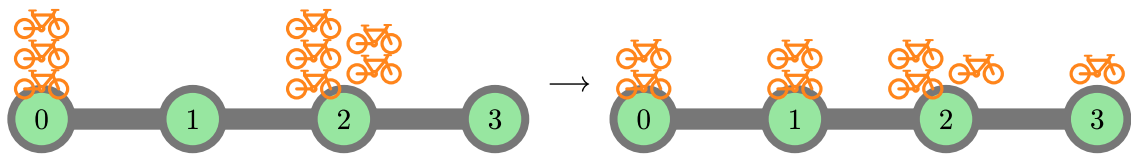
移動距離の合計は 6 である．この戦略に対応する返り値は $X = [0, 2, 1, 2, 4, 2, 3]$ と $Y = [-1, -1, 2, 0, -2, 1, 1]$ である．この戦略が最小の移動距離を達成することが証明できる．よって，関数は $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$ を返すべきである．

移動距離が 6 である他の正当な戦略，例えば $X = [0, 2, 3, 2, 4, 2, 1]$ と $Y = [-1, 0, 1, 0, -2, 0, 2]$ も正答である．長さが $7 = 6 + 1$ であるが正当な戦略ではない配列 X, Y ，例えば $X = Y = [0, 0, 0, 0, 0, 0, 0]$ を返した場合，得点の 50% を得る．

例 3

以下の呼び出しを考える．

```
find_rebalancing_strategy(4,
                        [3, 0, 5, 0],
                        [2, 2, 3, 1],
                        [0, 1, 2],
                        [1, 2, 3])
```



最適解は $X = [2, 1, 0, 1, 2, 3]$ と $Y = [-1, 1, -1, 1, -1, 1]$ である．関数は $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$ を返すべきである．他の正当な戦略，例えば $X = [2, 1, 0, 1, 2, 3]$ と $Y = [-2, 2, -1, 0, 0, 1]$ も正答である．

この例は小課題 2 と 3 の制約を満たしている．

採点プログラムのサンプル

入力の最初の行は，シナリオの数を表す 1 つの整数 T からなる必要がある． T 個のシナリオの記述は，それぞれ以下で指定された形式に従う必要がある．

入力形式：

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

出力形式：

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```